

CSCE 611 OPERATING SYSTEMS · MODULE 08 · LECTURE NOTES

File Systems

Turning a block device into names, directories, and bytes.

MODULE OUTCOMES

- Explain inodes and the separation of names from files.
- Compare block allocation strategies and their costs.
- Explain how directories and path resolution work.
- Explain hard links, symlinks, and why unlink is named as it is.
- Explain the buffer cache and why fsync exists.

1 The layered structure

A disk presents a flat array of numbered fixed-size blocks. A file system builds, in layers, the illusion of named growable byte arrays arranged in a hierarchy.

Layer	Input	Output
Block device	—	An array of 512- or 4096-byte blocks.
Buffer cache	Blocks	Cached blocks; write buffering and reordering.
Inode layer	Blocks	A growable byte array with metadata.
Directory layer	Inodes	A mapping from names to inode numbers.
Path layer	Directories	Hierarchical path resolution.
Descriptor layer	Inodes	Open files with per-descriptor offsets, shared across fork and dup.

THE INODE IS THE FILE; THE NAME IS NOT

A file's metadata and block pointers live in an **inode**, identified by a number. A **directory** is simply a file whose contents are (name, inode number) pairs.

So a filename is a *reference* to a file rather than the file itself. One file may have several names, and removing a name does not necessarily remove the file.

Once this separation is clear, three otherwise confusing behaviours become obvious: hard links, the fact that the deletion call is named unlink, and a running program continuing to work after its executable is deleted.

2 Finding a file's blocks

Scheme	Representation	Problem
Contiguous	Start block and length.	Reading is maximally efficient. The file cannot grow without moving, and the free space fragments externally — exactly the Module 06 problem.
Linked	Each block stores the number of the next.	Growth is free; random access is $O(n)$, and losing one pointer loses the entire remainder of the file.
File allocation table	The links extracted into one array.	FAT. Random access improves if the table is in memory, which means the whole table must be, and it scales poorly.
Indexed (inode)	A table of block numbers per file.	What essentially everything uses. $O(1)$ random access and free growth.
Extents	(start, length) runs instead of individual blocks.	Far more compact for large, contiguously allocated files. Used by ext4, XFS, NTFS, APFS.

2.1 Direct and indirect blocks

```
inode:
  type, size, link count, permissions, timestamps
  addrs[0..11]    -> 12 direct block pointers
  addrs[12]       -> singly indirect (a block of pointers)
  [ext2 also: doubly indirect, triply indirect]
```

With 1 KB blocks and 4-byte pointers (256 pointers per block):

```
direct      12 blocks = 12 KB
singly indirect 256 blocks = 256 KB
doubly indirect 256 x 256 = 64 MB
triply indirect 256^3     = 16 GB
```

The design is a deliberate optimisation for the observed distribution of file sizes. Most files are small, and for those every data block is reachable directly from the inode — one inode read and then the data. Large files pay additional indirection, but random access remains $O(1)$ with a bounded small constant.

Modern file systems prefer **extents**, which record runs of contiguous blocks rather than individual numbers. A 1 GB contiguously allocated file needs one extent rather than a quarter of a million pointers, which also makes the metadata itself cache-friendly.

3 Names and directories

Resolving `/usr/bin/ls`:

1. Begin at the root inode (a fixed, known inode number).
2. Read the root directory's data; search for "usr" -> inode 42.
3. Read inode 42; verify it is a directory; read its data.
4. Search for "bin" -> inode 87.
5. Read inode 87; read its data; search for "ls" -> inode 1203.
6. Return inode 1203.

Every path component costs at least one inode read and one data block read, and a permission check. Deep paths are therefore genuinely expensive, which is why kernels maintain a **dentry cache** mapping recently resolved (directory, name) pairs straight to inodes.

Note that the permission checked at each component is **execute** on the directory, which for a directory means 'may traverse'. This is why a directory can be traversable but not listable (`--x`): you can open a file inside it if you know the name, and you cannot enumerate the contents.

3.1 Hard links and symbolic links

	Hard link	Symbolic link
What it is	An additional directory entry pointing at the same inode.	A small file whose contents are a path string.
Resolution	None needed — it is the inode number.	The path is resolved on every access.
Original vs copy	There is no original. All names are equal.	There is a clear target, which may not exist.
Link count	Incremented in the inode.	Not affected.
Restrictions	Cannot cross file systems (inode numbers are per-filesystem); cannot link directories (would permit cycles).	No restrictions. May point anywhere, including nowhere.
If the target goes	Nothing happens — the file survives while any name remains.	The link dangles.

WHY THE SYSTEM CALL IS CALLED UNLINK

`unlink(path)` removes a *name* and decrements the inode's link count. It does not necessarily delete anything. The file's blocks are released only when the link count reaches zero **and** no process holds it open. The kernel tracks both.

This produces two familiar behaviours. First, deleting a file that a running program has open is harmless — the program continues reading it, and the space is reclaimed when the last descriptor closes. Programs exploit this deliberately for temporary files: create, unlink immediately, and the file is guaranteed to vanish when the process exits however it exits.

Second, the classic operations puzzle: `df` reports the disk full, `du` accounts for far less, and the space reappears when a process is restarted — because it was holding a deleted log file open.

4 The buffer cache

Disk blocks are cached in memory. Reads check the cache first; writes modify the cached copy and return immediately, with the actual device write deferred.

Benefit	Mechanism
Reads avoid the device entirely.	Cache hits on recently used blocks — locality applies here exactly as in CSCE 614 Module 05.
Repeated writes collapse.	Ten writes to the same block within a second become one device write.
Writes are batched and reordered.	The kernel can issue them in an order that suits the device — historically elevator ordering for seek time, now mostly queue depth management for SSDs.
Read-ahead.	Sequential access is detected and subsequent blocks are fetched before they are requested.

WRITE() DOES NOT MEAN WRITTEN

A successful `write()` guarantees that the data is in the kernel's page cache. It guarantees nothing about durable storage.

The data may sit there for tens of seconds. A power failure, kernel panic, or disconnected cable loses it, and *no error is reported to anybody* — the write already returned success.

`fsync(fd)` forces the file's data to durable media and does not return until the device reports completion. It is the only guarantee available, and it is roughly a thousand times slower than a cached write.

And it is not sufficient by itself. Creating a file involves writing the file's data *and* adding its name to a directory. `fsync` on the file does not flush the directory, so after a crash the file's blocks can exist with no name referring to them. Durably creating a file means `fsyncing` the file and then the containing directory. Module 09 develops why this ordering matters.

Operation	Typical latency	What is guaranteed
<code>write()</code> into the page cache	~1 μ s	Nothing durable. The data is in volatile memory.
<code>fsync()</code> to an SSD	100 μ s – 1 ms	Data has reached the device's durable storage.
<code>fsync()</code> to a rotating disk	5–10 ms	As above, after a seek and a rotation.
<code>fsync()</code> to a device with a lying write cache	Fast	Nothing. Some consumer drives acknowledge before data is durable. This is documented, real, and the cause of a great deal of 'impossible' corruption.

The three-orders-of-magnitude gap between a cached write and a durable one is why databases and file systems work so hard to minimise synchronous writes, and why the next module exists.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 39–41 (Files and Directories, File System Implementation, FFS)	Inodes, directories, and allocation, with the Fast File System's locality arguments.
xv6 book — Chapter 8 (File system)	A complete, readable file system in about a thousand lines, organised in exactly the layers of §1.
MIT 6.1810 — file system lectures and lab	Including the lab that adds large files and symbolic links.
Linux Documentation — ext4 and VFS	How extents, delayed allocation, and the virtual file system layer work in production.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read xv6's `fs.c` and map each function to a layer in §1's table.
- 2 Add doubly indirect blocks to xv6's inode and verify you can create a file larger than the previous maximum.
- 3 Implement symbolic links in xv6, including loop detection for a symlink chain that points to itself.
- 4 Create a hard link to a file, modify through one name, and read through the other. Then remove one name and confirm the data survives. Report the link count at each step.
- 5 Open a file, unlink it while keeping the descriptor, write to it, and read it back. Check `df` and `du` while the descriptor is open, then close it and check again.
- 6 Measure the cost of durability: time ten thousand `write()` calls, then ten thousand `write() + fsync()` pairs. Report the ratio.
- 7 Write a program that creates a file and `fsyncs` only the file. Crash the machine (or use a VM snapshot) and determine whether the file exists afterwards. Then `fsync` the directory too and repeat.
- 8 Instrument path resolution in xv6 to count block reads for `/a/b/c/d/e`. Explain the number.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is an inode, and in what sense is a filename not the file?
- 2 Compare contiguous, linked, and indexed allocation, giving the specific failure of each of the first two.

- 3 Why does an inode have both direct and indirect block pointers?
- 4 Describe path resolution and say which permission is checked at each component.
- 5 Distinguish hard and symbolic links, and explain why hard links cannot cross file systems.
- 6 Why is the call named unlink, and what two conditions must hold before blocks are freed?
- 7 What exactly does a successful write() guarantee, and what must you do for durability?